

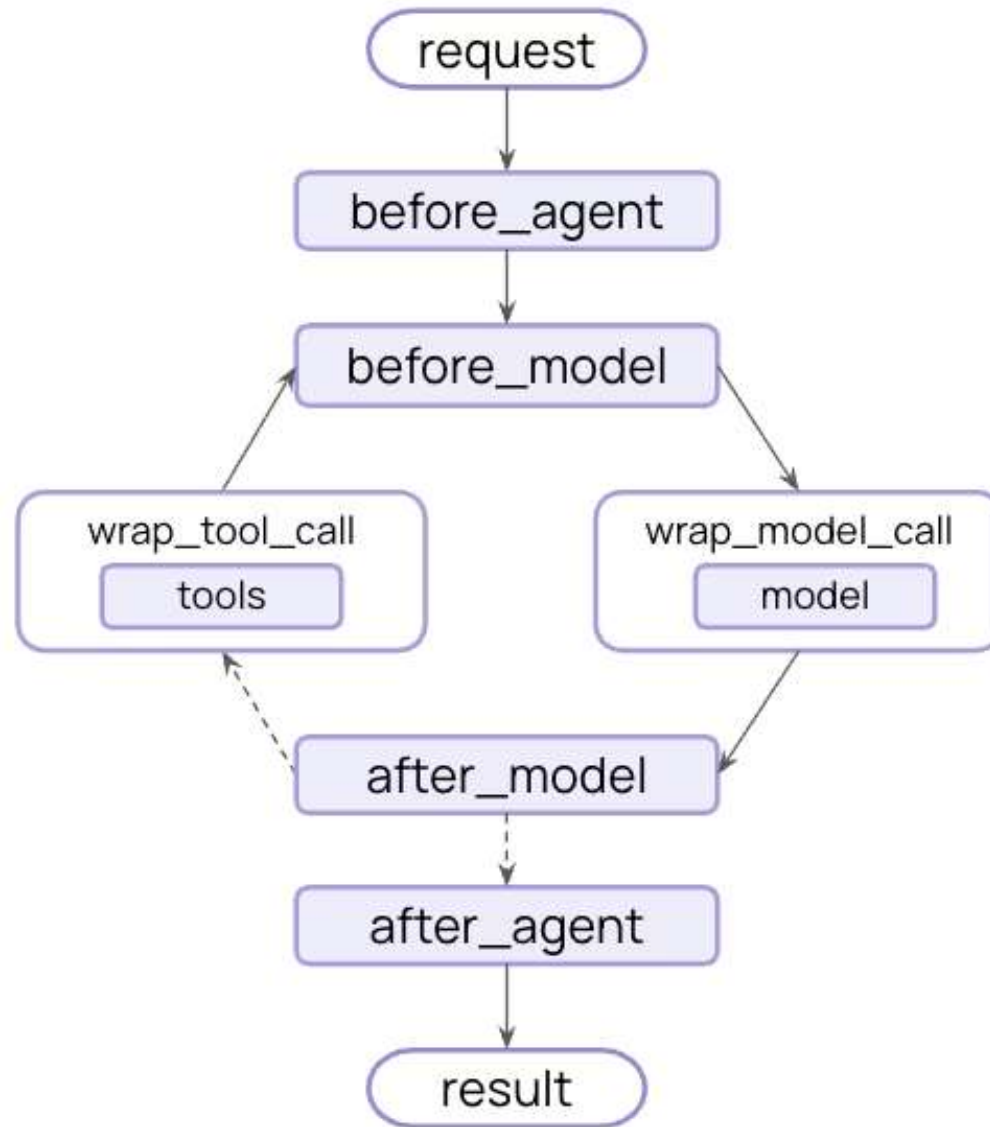
Guardrails



Guardrails

- Programmatic mechanisms designed to ensure the AI operates strictly within defined boundaries
- Guardrails typically monitor:
 - **Inputs:** Protecting against prompt injections or filtering PII before it reaches the model.
 - **Outputs:** Ensuring structural integrity (like valid JSON or specific schemas), verifying factual accuracy, or blocking toxic/inappropriate content.
 - **Execution:** Preventing infinite loops or unauthorized tool usage.

Langchain Guardrails



Guardrail Patterns

1. Deterministic guardrails

- Rule-based checks such as regex, keyword matching, allow/deny lists, schema checks
- Fast, predictable, and low cost.
- May miss some semantic violations.

2. Model-based guardrails

- Use an LLM or classifier to judge safety, quality, or compliance.
- Better for subtle or contextual risks
- Slower and more expensive than deterministic checks.

Built-in guardrails

- **Personally Identifiable Information (PII) Detection**

- LangChain provides built-in middleware for detecting and handling **PII** in conversations.
- This middleware can detect common PII types like emails, credit cards, IP addresses, and more.
- The PII middleware supports multiple strategies for handling detected PII

Strategy	Description	Example
redact	Replace with [REDACTED_{PII_TYPE}]	[REDACTED_EMAIL]
mask	Partially obscure (e.g., last 4 digits)	****_****_****-1234
hash	Replace with deterministic hash	a8f5f167...
block	Raise exception when detected	Error thrown

PII Middlewa re

```
agent = create_agent(  
    model="gpt-5.4",  
    tools=[customer_service_tool, email_tool],  
    middleware=[  
        # Redact emails in user input before sending to model  
        PIIMiddleware(  
            "email",  
            strategy="redact",  
            apply_to_input=True,  
        ),  
        # Mask credit cards in user input  
        PIIMiddleware(  
            "credit_card",  
            strategy="mask",  
            apply_to_input=True,  
        ),  
        # Block API keys - raise error if detected  
        PIIMiddleware(  
            "api_key",  
            detector=r"sk-[a-zA-Z0-9]{32}",  
            strategy="block",  
            apply_to_input=True,  
        ),  
    ],  
)
```

Custom Guardrails

```
@before_agent(can_jump_to=["end"])
def content_filter(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    """Deterministic guardrail: Block requests containing banned keywords."""

    # Get the first user message
    if not state["messages"]:
        return None

    first_message = state["messages"][0]
    if first_message.type != "human":
        return None

    content = first_message.content.lower()
    # Check for banned keywords
    for keyword in banned_keywords:
        if keyword in content: # Block execution before any processing
            .
            .
```

Custom Guardrails (cont...)

```
from langchain.agents import create_agent

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, calculator_tool],
    middleware=[content_filter],
)
```


Implementing Guardrails in LangGraph

Use Validation Nodes

- Incorporate validation nodes within the architecture to verify and validate data or model outputs at various points in the graph.

Implement Model Fallbacks

- Set up fallback mechanisms that activate when a primary model's output fails validation or confidence thresholds are not met.

Human-in-the-Loop (HITL) Integration

- Involve human oversight by integrating HITL processes where human judgment can review, approve, or correct outputs as needed.

Validation Nodes

```
workflow = StateGraph(GraphState)
workflow.add_node("generate_response", generate_response)
workflow.add_node("validate_output", validate_output)

workflow.set_entry_point("generate_response")
workflow.add_edge("generate_response", "validate_output")

# The Guardrail Conditional Edge
workflow.add_conditional_edges(
    "validate_output", route_validation
)
```

Validation Nodes (cont...)

```
def validate_output(state: GraphState):  
    last_message = state["messages"][-1].content  
  
    # Example Check:  
    # Ensure the output doesn't contain restricted financial advice  
    if "guaranteed return" in last_message.lower():  
        return {  
            "is_valid": False,  
            "validation_feedback":  
                "Do not promise guaranteed returns.\  
                Use compliant language."  
        }  
  
    return {"is_valid": True, "validation_feedback": ""}
```

Model Fallback

```
from langchain.agents import create_agent
from langchain.agents.middleware import ModelFallbackMiddleware

agent = create_agent(
    model="gpt-5.4",
    tools=[],
    middleware=[
        ModelFallbackMiddleware(
            "gpt-5.4-mini",
            "claude-3-5-sonnet-20241022",
        ),
    ],
)
```

Model & Tool Fallback

```
# Primary model (maybe cheaper/faster)
primary_llm = ChatOpenAI(model="gpt-3.5-turbo")

# Fallback model (smarter, better at formatting)
backup_llm = ChatAnthropic(model="claude-3-opus-20240229")

# Bind tools
tools = [execute_sql_query, fetch_user_data]
primary_llm_with_tools = primary_llm.bind_tools(tools)
backup_llm_with_tools = backup_llm.bind_tools(tools)

# Create the fallback chain
# If the primary LLM fails (e.g., OutputParserException), it tries the backup
robust_llm = primary_llm_with_tools.with_fallbacks([backup_llm_with_tools])

# Inside your LangGraph Node:
def agent_node(state: GraphState):
    # This call is now protected by a fallback guardrail
    response = robust_llm.invoke(state["messages"])
    return {"messages": [response]}
```

Combining Multiple Guardrails

```
agent = create_agent(  
    model="gpt-5.4",  
    tools=[search_tool, send_email_tool],  
    middleware=[  
        # Layer 1: Deterministic input filter (before agent)  
        ContentFilterMiddleware(banned_keywords=["hack", "exploit"]),  
  
        # Layer 2: PII protection (before and after model)  
        PIIMiddleware("email", strategy="redact",  
            apply_to_input=True),  
        PIIMiddleware("email", strategy="redact",  
            apply_to_output=True),  
  
        # Layer 3: Human approval for sensitive tools  
        HumanInTheLoopMiddleware(interrupt_on={"send_email": True}),  
  
        # Layer 4: Model-based safety check (after agent)  
        SafetyGuardrailMiddleware(),  
    ],
```